

Engineer Onboarding Handbook

Synthetic Dataset — Edition 02 • Fictional organization and resources

Welcome to Meridian Systems! We build software products for logistics, operations, analytics, and customer experience. This handbook gives new engineers a practical starting point for understanding our teams, projects, tools, development practices, and first-month expectations.

Data disclaimer: Every person, project, handle, URL, and organizational detail in this document is synthetic. It is intended for testing, demonstrations, and training only.

1. Welcome to Meridian Systems

Your first few weeks are focused on learning the environment rather than immediately becoming an expert. You will receive access gradually, meet the people who support your work, and complete a small first contribution.

What Success Looks Like

- You can navigate the repository and engineering documentation.
- You understand the team's branching, review, testing, and release workflow.
- You know who to contact for technical, product, design, QA, and infrastructure questions.
- You can run your assigned service locally and make a small safe change.
- You communicate blockers clearly and ask for help when needed.

2. Engineering Organization

Meridian Systems uses cross-functional product squads supported by shared platform teams.

Position	Person	Handle	Area
VP Engineering	Maya Srinivasan	@maya-s	Engineering organization
Platform Manager	Daniel Fernandes	@daniel-f	Infrastructure and platform
Backend Lead	Farhan Ali	@farhan-a	Services and APIs
Web Lead	Pooja Menon	@pooja-m	Web applications
QA Manager	Sameer Kulkarni	@sameer-k	Quality engineering
SRE Lead	Laura Thomas	@laura-t	Reliability and observability
Product Design Lead	Ira Banerjee	@ira-b	UX/UI and design systems
Data Lead	Nitin Bhat	@nitin-b	Analytics and data platforms

How to Get Help

- **Mentor:** onboarding questions, context, and day-to-day guidance.
- **Project lead:** feature requirements, priorities, and technical decisions.
- **QA:** test strategy, regressions, and release validation.
- **SRE/Platform:** deployments, infrastructure, observability, and production issues.
- **Design:** UI behavior, accessibility, and product experience.

3. Current Product Portfolio

Project	Description	Area	Lead	Status
RouteFlow	Shipment routing and dispatch management	Backend	Farhan Ali	Active

Project	Description	Area	Lead	Status
Harbor	Operations dashboard for logistics teams	Web	Pooja Menon	Active
Signal	Real-time event and alerting platform	Platform	Daniel Fernandes	Active
LedgerX	Internal billing reconciliation service	Backend/Data	Farhan Ali	Planning
Vista	Executive analytics and reporting	Data	Nitin Bhat	Active
Compass	Customer support workspace	Web	Pooja Menon	Maintenance
Anchor	Infrastructure provisioning toolkit	SRE	Laura Thomas	Active
Relay	Internal messaging and notification service	Platform	Daniel Fernandes	Planning

Project Assignment Process

Your manager and project lead will assign you to one primary squad. During the first week, you should focus on understanding the product domain and architecture before selecting larger tasks.

- Read the project README.
- Review the architecture diagram.
- Understand the main services or application modules.
- Run the project locally.
- Review three recent pull requests.
- Identify the team's main test commands.

4. Synthetic Engineer Directory

ID	Engineer	Role	Project	Level	Mentor
MS-201	Vihaan Joshi	Backend Engineer	Platform	Senior	Farhan Ali
MS-202	Sana Kapoor	Frontend Engineer	Harbor	Mid-Level	Pooja Menon
MS-203	Reyansh Sethi	Data Engineer	Vista	Junior	Nitin Bhat
MS-204	Diya Chawla	QA Engineer	Signal	Mid-Level	Sameer Kulkarni
MS-205	Arnav Bedi	SRE Engineer	Anchor	Senior	Laura Thomas
MS-206	Myra Saxena	Full-Stack Engineer	RouteFlow	Mid-Level	Pooja Menon
MS-207	Ayaan Dutta	Backend Engineer	Relay	Junior	Farhan Ali
MS-208	Kiara Bose	Data Analyst	Vista	Mid-Level	Nitin Bhat
MS-209	Dhruv Anand	Platform Engineer	Signal	Senior	Daniel Fernandes
MS-210	Anvi Menon	Frontend Engineer	Compass	Junior	Pooja Menon
MS-211	Kabir Arora	Backend Engineer	LedgerX	Mid-Level	Farhan Ali
MS-212	Tara Pillai	QA Automation Engineer	RouteFlow	Mid-Level	Sameer Kulkarni

5. Development Environment

New engineers should use the standard workstation configuration where possible. Project-specific requirements are documented in each repository.

Required Tools

- Git for version control
- A supported code editor or IDE
- Python 3.12 or the project's specified runtime

- Node.js LTS for web projects
- Docker Desktop or the approved container runtime
- PostgreSQL client tools when required
- Company-approved password manager

Initial Setup

```
git clone https://example.com/meridian/routeflow.git
cd routeflow
git checkout -b feature/first-task
make setup
make test
```

The commands above are illustrative. Replace them with the commands in your assigned project's README.

6. First Week Schedule

Day	Theme	Planned Activities
Monday	Orientation	HR session, access requests, workstation setup, mentor introduction.
Tuesday	Product & Stack	Product walkthrough, architecture overview, technology stack session.
Wednesday	Project Immersion	Repository tour, local setup, issue-tracker walkthrough.
Thursday	Engineering Practices	Testing, pull requests, CI/CD, incident process, coding standards.
Friday	First Contribution	Small task, mentor review, retrospective, manager check-in.

7. Onboarding Checklist

Access and Administration

- Complete HR onboarding forms.
- Activate company account and multi-factor authentication.
- Request repository access.
- Request issue-tracker access.
- Join required engineering channels.
- Complete security awareness training.

Technical Setup

- Install Git and configure SSH or approved authentication.
- Install the supported IDE.
- Install project dependencies.
- Configure local environment variables.
- Run unit tests successfully.
- Run the application or service locally.
- Verify logs and health checks.

Team Integration

- Meet your mentor.
- Meet your project lead.
- Attend at least one stand-up.

- Attend one sprint or planning session.
- Review an existing pull request.
- Open your first draft pull request.

8. Git and Pull Request Standards

All production code changes should go through version control and peer review. The goal of review is to improve correctness, maintainability, security, and shared understanding.

- **Branch:** Keep branches focused on one change.
- **Commit:** Write messages that explain the change.
- **Pull request:** Explain what changed, why it changed, and how it was tested.
- **Review:** Respond to comments and resolve discussions before merging.
- **Merge:** Do not bypass required checks or approvals.

Example Pull Request Template

Summary

Describe the problem and solution.

Testing

List commands or validation performed.

Risks

Mention migrations, compatibility concerns, or rollout considerations.

Follow-up

List known future improvements.

9. Testing and Quality

Meridian follows a layered testing approach. The appropriate level depends on the change.

Test Type	Purpose	Typical Owner
Unit	Validate individual functions/components	Developer
Integration	Validate service or database interactions	Developer / QA
API	Validate contracts and endpoint behavior	Developer / QA
End-to-End	Validate critical user journeys	QA / Developer
Regression	Ensure existing behavior remains correct	QA
Load	Evaluate behavior under expected traffic	Platform / QA

Definition of Done

- Implementation is complete and readable.
- Relevant tests have been added or updated.
- Linting and formatting checks pass.
- Documentation has been updated where necessary.
- Pull request has received required reviews.
- No known critical issue remains unresolved.

10. Deployment and Reliability

Production deployments are automated wherever possible. Engineers should understand how a change moves from a branch to a production release.

- Continuous integration runs automated checks on pull requests.
- Approved changes are promoted through staging before production.

- Monitoring dashboards should be checked after significant releases.
- Production incidents follow a documented response process.
- Rollback procedures should be understood before high-risk releases.

Incident Response Basics

- Confirm the issue and assess its impact.
- Notify the responsible team or on-call engineer.
- Record important observations in the incident channel.
- Avoid unreviewed high-risk changes during an active incident.
- Document the final resolution and follow-up actions.

11. Security Expectations

- Never commit passwords, tokens, private keys, or API credentials.
- Use synthetic or approved test data in development.
- Do not paste confidential customer information into public tools.
- Use least-privilege access.
- Report suspected security incidents immediately.
- Follow dependency and vulnerability-management procedures.

12. Communication Standards

Good Technical Questions

When asking for help, include: **context** → **expected behavior** → **actual behavior** → **what you tried** → **relevant logs or error messages**.

Example

Context: The RouteFlow API fails during local startup.
Expected: Service starts on port 8000.
Actual: Database connection refused.
Tried: Restarted the database container and checked environment variables.
Question: Is there a required local database seed command?

13. Team Channels

- **#meridian-engineering** — organization-wide engineering updates
- **#help-development** — development questions
- **#routeflow** — RouteFlow project discussions
- **#harbor** — Harbor project discussions
- **#platform** — infrastructure and shared platform topics
- **#quality-engineering** — testing and quality topics
- **#incident-response** — active reliability incidents

14. Mentor Program

Mentor	Specialty	Experience	Primary Support
Farhan Ali	Backend / APIs	8 years	Services, architecture, code reviews

Mentor	Specialty	Experience	Primary Support
Pooja Menon	Frontend	5 years	React, accessibility, UI patterns
Sameer Kulkarni	QA	9 years	Automation, regression, releases
Laura Thomas	SRE	7 years	Reliability, monitoring, deployments
Nitin Bhat	Data	6 years	SQL, pipelines, analytics
Daniel Fernandes	Platform	10 years	Cloud infrastructure and tooling

Suggested Check-ins

- Day 1 — introductions and onboarding goals
- Day 3 — project and technical questions
- Day 5 — first-week review
- Week 2 — first contribution feedback
- Week 4 — progress and development goals

15. 30-Day Development Plan

Period	Focus	Expected Outcome
Days 1–5	Orientation and environment	Accounts, tools, team context, local setup
Week 2	Project learning	Understand architecture and complete small fixes
Week 3	Independent contribution	Deliver a reviewed engineering task
Week 4	Ownership	Own a small feature or improvement and present learnings

16. Frequently Asked Questions

I cannot get the project running. What should I do?

Read the setup instructions, verify prerequisites, inspect the error message, and ask in the development-help channel with the context of what you tried. Your mentor can help if the issue remains blocked.

Should I ask before making a code change?

For a small assigned ticket, follow the ticket and project conventions. For architectural or potentially risky changes, discuss the approach with your project lead before implementation.

How do I know whether a task is urgent?

Use the priority and severity defined by the project. Production incidents and customer-impacting failures take precedence over normal feature work.

Where should I document what I learn?

Put reusable technical knowledge in the project documentation or engineering knowledge base rather than keeping important information only in private messages.

17. End-of-Month Review

- I understand my project's purpose and major components.
- I can run the project locally.
- I understand the Git and pull-request workflow.
- I have written or updated tests.

- I have completed at least one reviewed engineering contribution.
- I know the appropriate contacts for technical and operational issues.
- I have discussed development goals with my manager.

Final reminder: You are not expected to know everything during onboarding. The goal is to build enough context and confidence to contribute safely, communicate effectively, and keep learning.

18. Welcome to Meridian Systems

We're happy to have you on the team. Build thoughtfully, review carefully, ask questions early, and help make the engineering environment better for the next person who joins.

Meridian Systems Engineering • Synthetic Edition 02 • 2026